Practica 5

Práctica 5 - Transporte 1.pdf

1. **¿Cuál es la función de la capa de transporte?**

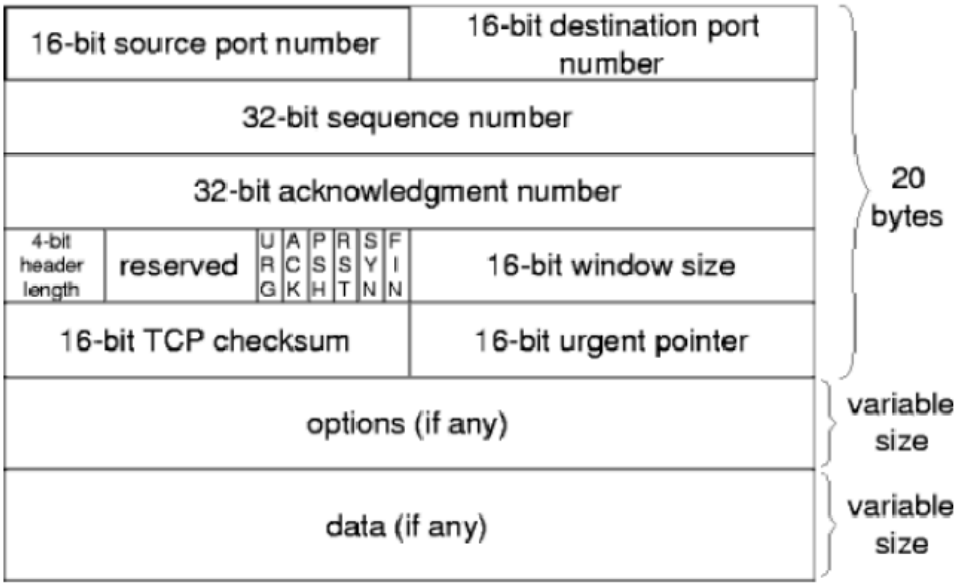
   -La funcion principal es proporcionar una comunicacion logica entre procesos de aplicacion que se ejecutan en dispositivos diferentes dentro de una red. Esta comunicacion permite que los procesos se envien mensajes entre si sin preocuparse por la infraestructura fisica de por medio

   -Los protocolos de la capa de transporte se implementan en los sistemas terminales hosts y no en los routers de la red. Por parte del emisor, la capa de transporte convierte los mensajes de la capa de aplicacion en segmentos de la capa de transporte, luego se encapsulan en paquetes de la capa de red y se envian a destino. Los routers solo actuan sobre los campos correspondientes a la capa de red del paquete, sin examinar campos del segmento de la capa de transporte encapsualdo. Por parte del receptor, la capa de transporte extraer el segmento de la capa de transporte del paquete y lo entrega a la aplicacion receptora. Para las aplicaciones de red, existen vairos protocolos de la capa de transporte disponibles (TCP y UDP), los cuales ofrecen caracteristicas diferentes y servicios a las aplicaciones que los utilizan

2. **Describa la estructura del segmento TCP y UDP.**

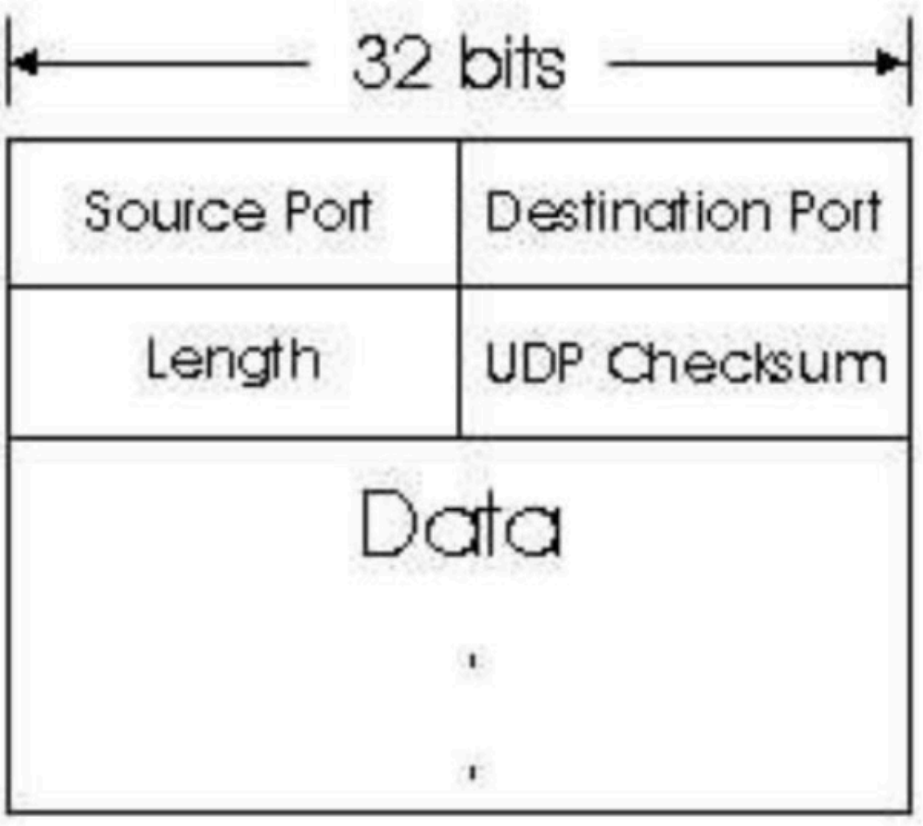
   -TCP:

TCP	
16-bit source port number	Indica el número de puerto de origen de la aplicación que está enviando los datos.
16-bit destination port number	Especifica el número de puerto de destino de la aplicación receptora.
32-bit sequence number	Este campo se utiliza para mantener un seguimiento del orden de los segmentos TCP en una comunicación. Cada segmento TCP se etiqueta con un número de secuencia único.
32-bit acknowledgment number	Indica el número de secuencia que espera recibir el emisor del siguiente segmento. Ayuda a establecer que los datos se han recibido de manera confiable.
4-bit header length	Este campo especifica la longitud del encabezado TCP en palabras de 32 bits. Se utiliza para identificar dónde comienza la carga útil de datos en el segmento.
reserved	Este campo se reserva para uso futuro y debe establecerse en cero.
Flags	Son bits que se utilizan para controlar y gestionar la comunicación entre los dispositivos.
16-bit widow size	Indica el tamaño de la ventana de recepción que el receptor tiene disponible para aceptar datos. Ayuda a controlar el flujo de datos en la conexión.
16-bit TCP checksum	Proporciona una suma de verificación para verificar la integridad de los datos en el segmento TCP y detectar errores de transmisión.
16-bit urgent pointer	Se utiliza en la comunicación para indicar la posición de datos urgentes dentro del segmento, si es necesario.
options	Este campo opcional permite la inclusión de información adicional en el encabezado TCP, como máximas segment size (MSS), ventana de escala, timestamp, entre otros. Las opciones se utilizan para ajustar y optimizar la comunicación según las necesidades de la aplicación.



-UDP:

UDP	
Source Port	Especifica el número de puerto del proceso que envía el datagrama UDP
Destination Port	Este campo indica el número de puerto del proceso de destino al que se debe entregar el datagrama UDP
Length	Este campo especifica la longitud total del datagrama UDP, incluyendo tanto el encabezado como los datos. La
	longitud se mide en bytes y permite al receptor conocer la cantidad de información que debe procesar en el datagrama.
UDP Checksum	La suma de verificación es un valor calculado que se utiliza para detectar errores en el datagrama UDP durante la transmisión. Se calcula en función del contenido del datagrama, incluyendo el encabezado y los datos. El receptor verifica esta suma de verificación para determinar si el datagrama UDP ha llegado intacto o si ha sufrido algún tipo de corrupción durante la transmisión.



3. **¿Cuál es el objetivo del uso de puertos en el modelo TCP/IP?**

   -Se usan para distinguir aplicaciones que estan enviando o recibiendo datos. Los puertos actuan como puntos finales en una comunicacion y permiten que multiples aplicaciones en una misma computadora o dispositivo se comuniquen simultaneamente a traves de la red

4. **Compare TCP y UDP en cuanto a:**

   a. **Confiabilidad**

TCP	UDP
Es un modelo confiable, garantiza que los datos se entreguen en el orden correcto y sin errores, ya que utiliza tecnicas como la retransmision de datos perdidos y deteccion y correcion de errores, usando sumas de verificaciones	UDP es menos confiable, no garantiza que los datos se entreguen y tampoco que lleguen en orden. Los segmentos pueden perderse y/o llegar desordenados sin correcion automatica

   b. **Multiplexación.**

TCP	UDP
Multiplexacion y demultiplexacion orientada a la conexion	Multiplexacion y demultiplexacion sin conexion
Socket TCP queda identifica por una tupal de 4 elementos: IP origen, Puerto origen, IP destino y Puerto destino. Por lo tanto, cuando un segmento TCP llega a un host procedente de la red, el host emplea los cuatros valores para	Socket UDP queda identificado por una tupla que consta de 2 elementos: IP destino y Puerto destino. Por lo tanto, si dos segmentos UDP tienen direcciones diferentes IP y/o Puertos de origen, pero la misma direccion IP destino

demultiplexar el segmento al socket apropiado	y el mismo puerto destino, entonces los dos segmentos se envian al mismo proceso destino a traves del mismo socket de destino

c. **Orientado a la conexión.**

TCP	UDP
Establece una conexion antes de la transmision de datos y asegura que ambas partes se sincronicen en terminos de secuencia de datos y control de flujos	No necesita una conexion para iniciar y finalizar una transmision de datos

d. **Controles de congestión.**

TCP	UDP
Proporciona mecanismos de control de congestion, los cuales evitan que cualquier conexion TCP inunde con una cantidad de trafico excesiva los enlaces y ruters existentes entre los hosts que estan comunicandose. Esto se consigue regulando la velocidad a la que los lados emisores de las conexiones TCP pueden enviar trafico a la red	Trafico UDP no esta controlado. Una aplicacion que use protocolo de transporte UDP puede enviar los datos a la velocidad que quiera, durante todo el tiempo que quiera
Cuenta con un mecanismo que indica aal emisor cuanto espacio libre hay en el buffer de almacenamiento del receptor (ventana), ayudando a controlar el flujo de datos para evitar la congestion y garantizar una comunicacion eficiente, permitiendo que el emisor ajuste la cantidad de datos enviados en funcion de la capacidad disponible del receptor	

e. **Utilización de puertos.**

TCP	UDP
Establece una conexxion punto a punto entre dos dispositivos, por lo que cada conexion esta limitada a dos procesos que intercambian datos	Permite que muchos clientes o procesos envien datos por el mismo socket
Usa numeros de puerto para identificar aplicaciones especificas	Usa numeros de puerto para identificar aplicaciones especificas

5. **La PDU de la capa de transporte es el segmento. Sin embargo, en algunos contextos suele utilizarse el término datagrama. Indique cuando.**

   -Cuando se usa UDP a esto se le suele llamar datagrama

6. **Describa el saludo de tres vías de TCP. ¿UDP tiene esta característica?**

   -El 3-way handshake es un metodo utilizado por TCP para establecer una conexion confiable entre dos dispositivos de una red. Requiere que tanto el cliente como el servidor intercambien segmentos SYN y ACK para que se establezca un acuerdo de ‘’llegue y ya me entere que

llegaste'' antes de que empiece la comunicacion de datos. Esto en UDP no existe ya que no se establece ninguna conexion

-Los pasos son:

-Paso 1 (SYN) → Cliente inicia el proceso enviando un segmento al servidor con el bit de SYN establecido y un numero de secuencia inicial (ISN) generado aleatoriamente. Es importante porque identifica y ordena los datos en la conexion

-Paso 2 (SYN/ACK) → Servidor recibe el segmento del cliente, reconoce el bit de SYN y responde enviando un segmento de respuesta con losl bits SYN y ACK estavlecidos. En este envio, el servidor tambien incluye su ISN elegido de manera aleatoria. Al mismo tiempo, el servidor reconoce el ISN del cliente, lo que indica que ha recibido correctamente el paquete de solicitud

-Paso 3 (ACK) → El cliente recibe la respuesta del servidor, reconociendo el ISN del mismo. Cliente responde enviando un segmento de confirmacion con el bit ACK estavlecido, confirmando que recibio la respuesta del servidor

7. **Investigue qué es el ISN (Initial Sequence Number). Relaciónelo con el saludo de tres vías.**

   -ISN (Initial Sequence Number) se refiere al numero de secuencia unico de 32 bits asignado a cada nueva conexion en una comunicacion TCP. Ayuda a que no entren en conflicto de bytes de datos transmitidos a travees de una conexion. El ISN es unico para cada conexion y esta separado para cada dispositivo. Ayuda a identificar, controlar el origen y mantener el orden de los segmentos transmitidos entre el cliente y servidor

8. **Investigue qué es el MSS. ¿Cuándo y cómo se negocia?**

   -En lugar de usar un tamanio de ventana de recepcion predeterminado de forma rigida, TCP ajusta a incrementos pares del tamanio maximo de segmentos (MSS). Maximun Segment Size es un campo de los encabezados que indica el tamanio mas grande datos que puede tener un segmento sin ser fragmentado. Mide la parte de un paquete que no tiene encabezado, lo que se conoce como carga util y esta determinado por una metrica que tiene que ver con el tamanio de los paquetes llamado MTU (Maximun Transer Unit), el cual incluye los encabezados TCP e IP (Protocolo de Internet)

   -MSS = MTU - (encabezado TCP + encabezado IP)

   -La diferencia entre MTU y MSS es que si un paquete supera la MTU de un dispositivo, se divide en partes mas chiquitas o se fragmenta. En cambio, si un paquete supera el MSS, se descarta y no se entrega. Por lo tanto, el MSS se negocia durante la configuracion de la conexion, es decir, durante el saludo de 3 vias

9. **Utilice el comando ss (reemplazo de netstat) para obtener la siguiente información de su PC:**

   a. **Para listar las comunicaciones TCP establecidas.**

   `ss -t`

   b. **Para listar las comunicaciones UDP establecidas.**

   `ss -u`

```
redes@debian:~$ ss -u
Recv-Q   Send-Q        Local Address:Port       Peer Address:Port     Process
0        0          10.0.2.15%enp0s3:bootpc         10.0.2.2:bootps
redes@debian:~$
```

c. **Obtener sólo los servicios TCP que están esperando comunicaciones**

`ss -t -l`

```
redes@debian:~$ ss -t -l
State   Recv-Q  Send-Q        Local Address:Port    Peer Address:Port  Process
LISTEN  0       128               0.0.0.0:ssh          0.0.0.0:*
LISTEN  0       128             127.0.0.1:ipp          0.0.0.0:*
LISTEN  0       5               127.0.0.1:4038         0.0.0.0:*
LISTEN  0       128                  [::]:ssh             [::]:*
LISTEN  0       128                [::1]:ipp              [::]:*
LISTEN  0       4096               [::1]:50051            [::]:*
LISTEN  0       4096     [::ffff:127.0.0.1]:50051            *:*
redes@debian:~$
```

d. **Obtener sólo los servicios UDP que están esperando comunicaciones.**

`ss -u -l`

```
redes@debian:~$ ss -u -l
State   Recv-Q   Send-Q     Local Address:Port      Peer Address:Port  Process
UNCONN  0        0               0.0.0.0:631          0.0.0.0:*
UNCONN  0        0             127.0.0.1:4038         0.0.0.0:*
UNCONN  0        0               0.0.0.0:51305        0.0.0.0:*
UNCONN  0        0               0.0.0.0:mdns         0.0.0.0:*
UNCONN  0        0                  [::]:42068           [::]:*
UNCONN  0        0                  [::]:mdns            [::]:*
redes@debian:~$
```

e. **Repetir los anteriores para visualizar el proceso del sistema asociado a la conexión.**

`ss -t -p ; ss -u -p ; ss -t -l -p ; ss -u -l -p`

```
root@debian:~# ss -t -p
State   Recv-Q   Send-Q     Local Address:Port     Peer Address:Port  Process
root@debian:~# ss -u -p
Recv-Q   Send-Q        Local Address:Port       Peer Address:Port     Process
0        0          10.0.2.15%enp0s3:bootpc         10.0.2.2:bootps    users:(("NetworkManager",pid=517,fd=23))
root@debian:~# ss -t -l -p
State    Recv-Q   Send-Q         Local Address:Port        Peer Address:Port   Process
LISTEN   0        128                  0.0.0.0:ssh              0.0.0.0:*       users:(("sshd",pid=605,fd=3))
LISTEN   0        128                127.0.0.1:ipp              0.0.0.0:*       users:(("cupsd",pid=586,fd=7))
LISTEN   0        5                  127.0.0.1:4038             0.0.0.0:*       users:(("core-daemon",pid=585,fd=5))
LISTEN   0        128                     [::]:ssh                 [::]:*       users:(("sshd",pid=605,fd=4))
LISTEN   0        128                   [::1]:ipp                  [::]:*       users:(("cupsd",pid=586,fd=6))
LISTEN   0        4096                  [::1]:50051                [::]:*       users:(("core-daemon",pid=585,fd=9))
LISTEN   0        4096       [::ffff:127.0.0.1]:50051                 *:*       users:(("core-daemon",pid=585,fd=10))
root@debian:~# ss -u -l -p
State     Recv-Q    Send-Q       Local Address:Port        Peer Address:Port    Process
UNCONN    0         0                  0.0.0.0:631              0.0.0.0:*       users:(("cups-browsed",pid=638,fd=7))
UNCONN    0         0                127.0.0.1:4038             0.0.0.0:*       users:(("core-daemon",pid=585,fd=6))
UNCONN    0         0                  0.0.0.0:51305            0.0.0.0:*       users:(("avahi-daemon",pid=514,fd=14))
UNCONN    0         0                  0.0.0.0:mdns             0.0.0.0:*       users:(("avahi-daemon",pid=514,fd=12))
UNCONN    0         0                     [::]:42068               [::]:*       users:(("avahi-daemon",pid=514,fd=15))
UNCONN    0         0                     [::]:mdns                [::]:*       users:(("avahi-daemon",pid=514,fd=13))
root@debian:~#
```

f. **Obtenga la misma información planteada en los items anteriores usando el comando netstat.**

`netstat -t -p ; netstat -u -p ; netstat -t -l -p ; netstat -u -l -p`

```
root@debian:~# netstat -t -p
Active Internet connections (w/o servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 10.0.2.15:51506         4.228.31.152:https      TIME_WAIT   -
tcp        0      0 10.0.2.15:51492         4.228.31.152:https      TIME_WAIT   -
tcp        0      0 10.0.2.15:51520         4.228.31.152:https      TIME_WAIT   -
tcp        0      0 10.0.2.15:51478         4.228.31.152:https      TIME_WAIT   -
root@debian:~# netstat -u -p
Active Internet connections (w/o servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
udp        0      0 10.0.2.15:bootpc        10.0.2.2:bootps         ESTABLISHED 517/NetworkManager
root@debian:~# netstat -t -l -p
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:ssh             0.0.0.0:*               LISTEN      605/sshd: /usr/sbin
tcp        0      0 localhost:ipp           0.0.0.0:*               LISTEN      586/cupsd
tcp        0      0 localhost:4038          0.0.0.0:*               LISTEN      585/python
tcp6       0      0 [::]:ssh                [::]:*                  LISTEN      605/sshd: /usr/sbin
tcp6       0      0 localhost:ipp           [::]:*                  LISTEN      586/cupsd
tcp6       0      0 localhost:50051         [::]:*                  LISTEN      585/python
tcp6       0      0 127.0.0.1:50051         [::]:*                  LISTEN      585/python
root@debian:~# netstat -u -l -p
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
udp        0      0 0.0.0.0:631             0.0.0.0:*                           638/cups-browsed
udp        0      0 localhost:4038          0.0.0.0:*                           585/python
udp        0      0 0.0.0.0:51305           0.0.0.0:*                           514/avahi-daemon: r
udp        0      0 0.0.0.0:mdns            0.0.0.0:*                           514/avahi-daemon: r
udp6       0      0 [::]:42068              [::]:*                              514/avahi-daemon: r
udp6       0      0 [::]:mdns               [::]:*                              514/avahi-daemon: r
root@debian:~#
```

10. **¿Qué sucede si llega un segmento TCP con el flag SYN activo a un host que no tiene ningún proceso esperando en el puerto destino de dicho segmento (es decir, el puerto destino no está en estado LISTEN)?**

    -Si no hay un proceso en estado LISTEN el host enviara un segmento con el flag RST (reset) activado para indicar que la conexion no se pudo establecer en dicho puerto. De esta forma, indica al emisor que no hay nadie y no no se puede establecer la conexion en ese momento, evitando que siga enviando segmentos

    a. **Utilice hping3 para enviar paquetes TCP al puerto destino 22 de la máquina virtual con el flag SYN activado.**

    `hping3 -S -p 22 localhost`

```
root@debian:~# hping3 -S -p 22 localhost
HPING localhost (lo 127.0.0.1): S set, 40 headers + 0 data bytes
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=0 win=65495 rtt=7.7 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=1 win=65495 rtt=3.8 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=2 win=65495 rtt=8.0 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=3 win=65495 rtt=7.3 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=4 win=65495 rtt=6.6 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=5 win=65495 rtt=6.0 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=6 win=65495 rtt=5.2 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=7 win=65495 rtt=0.4 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=8 win=65495 rtt=0.2 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=9 win=65495 rtt=8.2 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=10 win=65495 rtt=8.2 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=11 win=65495 rtt=3.1 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=12 win=65495 rtt=6.9 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=13 win=65495 rtt=5.4 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=14 win=65495 rtt=8.1 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=15 win=65495 rtt=8.0 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=16 win=65495 rtt=5.8 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=17 win=65495 rtt=3.9 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=18 win=65495 rtt=2.8 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=19 win=65495 rtt=5.2 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=20 win=65495 rtt=8.1 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=21 win=65495 rtt=7.2 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=22 win=65495 rtt=5.8 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=23 win=65495 rtt=5.2 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=24 win=65495 rtt=8.2 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=25 win=65495 rtt=7.5 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=26 win=65495 rtt=6.3 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=27 win=65495 rtt=4.9 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=28 win=65495 rtt=7.8 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=29 win=65495 rtt=7.0 ms
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=30 win=65495 rtt=5.2 ms
```

b. **Utilice hping3 para enviar paquetes TCP al puerto destino 40 de la máquina virtual con el flag SYN activado.**

`hping3 -S -p 40 localhost`

```
root@debian:~# hping3 -S -P 40 localhost
hping3: you must specify only one target host at a time
root@debian:~# hping3 -S -p 40 localhost
HPING localhost (lo 127.0.0.1): S set, 40 headers + 0 data bytes
len=40 ip=127.0.0.1 ttl=64 DF id=0 sport=40 flags=RA seq=0 win=0 rtt=3.8 ms
len=40 ip=127.0.0.1 ttl=64 DF id=0 sport=40 flags=RA seq=1 win=0 rtt=4.0 ms
len=40 ip=127.0.0.1 ttl=64 DF id=0 sport=40 flags=RA seq=2 win=0 rtt=6.9 ms
len=40 ip=127.0.0.1 ttl=64 DF id=0 sport=40 flags=RA seq=3 win=0 rtt=6.1 ms
len=40 ip=127.0.0.1 ttl=64 DF id=0 sport=40 flags=RA seq=4 win=0 rtt=4.4 ms
len=40 ip=127.0.0.1 ttl=64 DF id=0 sport=40 flags=RA seq=5 win=0 rtt=3.6 ms
len=40 ip=127.0.0.1 ttl=64 DF id=0 sport=40 flags=RA seq=6 win=0 rtt=2.0 ms
len=40 ip=127.0.0.1 ttl=64 DF id=0 sport=40 flags=RA seq=7 win=0 rtt=0.2 ms
len=40 ip=127.0.0.1 ttl=64 DF id=0 sport=40 flags=RA seq=8 win=0 rtt=4.5 ms
len=40 ip=127.0.0.1 ttl=64 DF id=0 sport=40 flags=RA seq=9 win=0 rtt=2.7 ms
len=40 ip=127.0.0.1 ttl=64 DF id=0 sport=40 flags=RA seq=10 win=0 rtt=5.0 ms
```

c. **¿Qué diferencias nota en las respuestas obtenidas en los dos casos anteriores? ¿Puede explicar a qué se debe? (Ayuda: utilice el comando ss visto anteriormente).**

-En el puerto 22 se tiene el flag SA, es decir SYN/ACK activados, lo que indica a que el puerto destino esta abierto y se pudo establecer una comunicacion. Por otro lado, en el puerto 40 se tiene el flag RA, es decir, RST/ACK, que indica que el puerto destino esta cerrado y no se pudo establecer una comunicacion

11. **¿Qué sucede si llega un datagrama UDP a un host que no tiene ningún proceso esperando en el puerto destino de dicho datagrama (es decir, que dicho puerto no está en estado LISTEN)?**

-Si no hay proceso en estado LISTEN, se respondera con el ICMP "Destination Unreachable'' (protoclo de IP). Este mensaje ICMP indica que el puerto o host de destino no estan disponibles, aunque puede perderse y UDP no generara ninguna notificacion al remitente ya que UDP es un protocolo sin conexion y poco confiable

1. **Utilice hping3 para enviar datagramas UDP al puerto destino 5353 de la máquina virtual.**

```
HPING localhost (lo 127.0.0.1): udp mode set, 28 headers + 0 data bytes
^C
--- localhost hping statistic ---
22 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
```

2. **Utilice hping3 para enviar datagramas UDP al puerto destino 40 de la máquina virtual.**

```
ICMP Port Unreachable from ip=127.0.0.1 name=localhost
status=0 port=2133 seq=2
ICMP Port Unreachable from ip=127.0.0.1 name=localhost
status=0 port=2134 seq=3
ICMP Port Unreachable from ip=127.0.0.1 name=localhost
status=0 port=2135 seq=4
ICMP Port Unreachable from ip=127.0.0.1 name=localhost
status=0 port=2136 seq=5
ICMP Port Unreachable from ip=127.0.0.1 name=localhost
status=0 port=2137 seq=6
ICMP Port Unreachable from ip=127.0.0.1 name=localhost
status=0 port=2138 seq=7
ICMP Port Unreachable from ip=127.0.0.1 name=localhost
status=0 port=2139 seq=8
```

3. **¿Qué diferencias nota en las respuestas obtenidas en los dos casos anteriores? ¿Puede explicar a qué se debe? (Ayuda: utilice el comando ss visto anteriormente).**

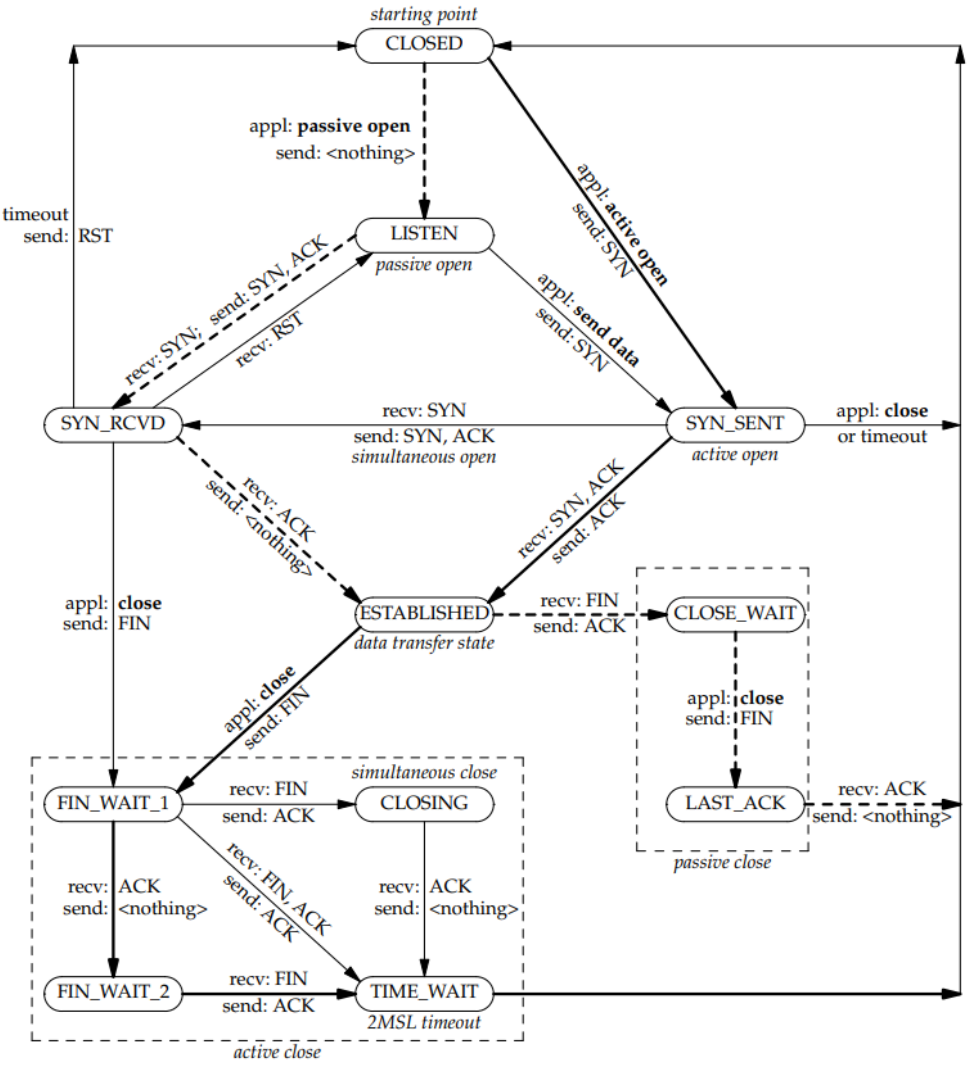
   -La diferencia es que en el puerto 5353 hay un proceso escuchando en estado de LISTEN y en el puerto 40 no. En el caso del puerto 40 se envia un ICMP que indica que el puerto es inalcanzable “Port Unreachable”.

   -Con respecto al comando, se puede ver que el puerto 5353 esta en estado activo

```
root@debian:~# ss -u -ln
State   Recv-Q   Send-Q     Local Address:Port      Peer Address:Port  Process
UNCONN  0        0              0.0.0.0:631            0.0.0.0:*
UNCONN  0        0            127.0.0.1:4038           0.0.0.0:*
UNCONN  0        0              0.0.0.0:51305          0.0.0.0:*
UNCONN  0        0              0.0.0.0:5353           0.0.0.0:*
UNCONN  0        0                 [::]:42068             [::]:*
UNCONN  0        0                 [::]:5353              [::]:*
root@debian:~#
```

12. **Investigue los distintos tipos de estado que puede tener una conexión TCP. Ver https://users.cs.northwestern.edu/~agupta/cs340/project2/TCPIP_State_Transition_Diagram**

    -Una conexion puede pasar por diferentes estados durante su ciclo de vida, dentro de los cuales se incluyen: LISTEN ; SYN-SENT ; SYN-RECEIVED ; ESTABLISHED ; FIN-WAIT 1 ; FIN-WAIT 2 ; CLOSE-WAIT ; CLOSING ; LAST-ACK ; TIME-WAIT ; y estado ficticio CLOSED. CLOSED es ficticio porque representa una situacion en la que no existe un bloque de control de transmision (TCB) lo que significa que no hay conexion alguna



-Lista de estados y sus significados:

- `LISTEN` → espera de una solicitud de conexion procedente de cualquier TCP remoto y puerto
- `SYN-SENT` → representa la espera de una solicitud de conexion coincidente despues de haber enviado una solicitud de conexion
- `SYN-RECEIVED` → representa la espera de una confirmacio nde solicitud de conexion despues de haber recibido y enviado una solicitud de conexion
- `ESTABLISHED` → representa una conexion abierta, permitiendo que los datos recibidos sean entregados al usuario. Es el estado normal durante la fase de transmision de datos
- `FIN-WAIT-1` → representa la espera de una solicitud de terminacion conexion procedente del TCP remoto, o la confirmacion de la solicitud de terminacion de conexion previamente enviada
- `FIN-WAIT-2` → representa la espera de una solicitud de terminacion procedente del TCP remoto
- `CLOSE-WAIT` → representa la espera de una solicitud de terminacion de conexion procedente del usuario local
- `CLOSING` → representa la espera de una confirmacion de la solicitud de terminacion de conexion procedente del TCP remoto

- `LAST-ACK` → representa la espera de una confirmacion de la solicitud de terminacion de conexion previamente enviada al TCP remoto (incluye una confirmacion de solicitud de terminacion de conexion)

- `TIME-WAIT` → representa la espera de suficiente tiempo para asegurar que el TCP remoto recibio la confirmacion de su solicitud de terminacion de cierre de conexion

- `CLOSED` → ausencia total de una conexion activa

13. **Dada la siguiente salida del comando ss, responda:**

```
Netid State      Recv-Q Send-Q Local Address:Port        Peer Address:Port       Process
tcp   LISTEN     0      128      *:22                          *:*          users:(("sshd",pid=468,fd=29))
tcp   LISTEN     0      128      *:80                          *:*          users:(("apache2",pid=991,fd=95))
udp   LISTEN     0      128      163.10.5.222:53               *:*          users:(("named",pid=452,fd=10))
tcp   ESTAB      0      0        163.10.5.222:59736          64.233.163.120:443       users:(("x-www-browser",pid=1079,fd=51))
tcp   CLOSE-WAI         T 0      0        163.10.5.222:41654             200.115.89.30:443
users:(("x-www-browser",pid=1079,fd=50))
tcp   ESTAB      0      0        163.10.5.222:59737          64.233.163.120:443       users:(("x-www-browser",pid=1079,fd=55))
tcp   ESTAB      0      0        163.10.5.222:33583           200.115.89.15:443       users:(("x-www-browser",pid=1079,fd=53))
tcp   ESTAB      0      0        163.10.5.222:45293           64.233.190.99:443       users:(("x-www-browser",pid=1079,fd=59))
tcp   LISTEN     0      128      *:25                          *:*                    users:(("postfix",pid=627,fd=3))
tcp   ESTAB      0      0        127.0.0.1:22              127.0.0.1:41220      users:(("sshd",pid=1418,fd=3),
                                                                                        ("sshd",pid=1416,fd=3))
tcp   ESTAB      0      0        163.10.5.222:52952     64.233.190.94:443       users:(("x-www-browser",pid=1079,fd=29))
tcp   TIME-WAIT  0      0        163.10.5.222:36676     54.149.207.17:443       users:(("x-www-browser",pid=1079,fd=3))
tcp   ESTAB      0      0        163.10.5.222:52960     64.233.190.94:443       users:(("x-www-browser",pid=1079,fd=67))
tcp   ESTAB      0      0        163.10.5.222:50521     200.115.89.57:443       users:(("x-www-browser",pid=1079,fd=69))
tcp   SYN-SENT   0      0        163.10.5.222:52132        43.232.2.2:9500      users:(("x-www-browser",pid=1079,fd=70))
tcp   ESTAB      0      0        127.0.0.1:41220           127.0.0.1:22         users:(("ssh",pid=1415,fd=3))
udp   LISTEN     0      128      127.0.0.1:53                  *:*           users:(("named",pid=452,fd=9))
```

a. **¿Cuántas conexiones hay establecidas?**

   -Hay 9 con estado ESTAB

b. **¿Cuántos puertos hay abiertos a la espera de posibles nuevas conexiones?**

   -4 puertos con estado LISTEN

c. **El cliente y el servidor de las comunicaciones HTTPS (puerto 443), ¿residen en la misma máquina?**

   -No residen en la misma maquina ya que IP local e IP origen son diferentes. Por ejemplo en la linea 4 la ip local es 163.10.5.222 y la IP destino es 64.233.163.120

d. **El cliente y el servidor de la comunicación SSH (puerto 22), ¿residen en la misma máquina?**

   -Si residen en la misma maquina. En la linea 8 la IP local es 127.0.0.1 y la IP destino es 127.0.0.1. La diferencia es que una esta en el puerto 22 y la otra en el 41220, la del puerto 22 esta corriendo un proceso SSHD y la del puerto 41220 (anteultima linea) esta corriendo un proceso SSH

e. **Liste los nombres de todos los procesos asociados con cada comunicación. Indique para cada uno si se trata de un proceso cliente o uno servidor.**

   -Procesos en LISTEN o en puertos del 0 al 1023 son procesos servidores:

   -SSHD → Servidor, escucha en el puerto *:22

   -apache2 → Servidor, escucha en el puerto de conexiones web *:80

   -named → Servidor, escucha en el puerto de peticiones DNS UDP *:53

-postfix → Servidor, escucha en el puerto de correos SMTP *25:

-x-www-browser → Cliente, navegador web que usa conexiones hacia puertos 443 y 9500

f. **¿Cuáles conexiones tuvieron el cierre iniciado por el host local y cuáles por el remoto?**

-TIME-WAIT (linea 12) → indica que el host local envio el primer paquete de finalizacion para cerrar la conexion y espera el suficiente tiempo para cerrar la conexion

-CLOSE-WAIT (linea 5) → indica que el host remoto envio el FIN pidiendo cerrar la conexion, el host local lo recibio y esta esperando a la aplicacion para que libere recursos de conexion

g. **¿Cuántas conexiones están aún pendientes por establecerse?**

-SYN-SENT (linea 15) → indica que se envio la solicitud de SYN pero todavia no se recibio el ACK del otro lado

14. **Dadas las salidas de los siguientes comandos ejecutados en el cliente y el servidor, responder:**

```
servidor# ss -natu | grep 110
tcp LISTEN 0 0 *:110 *:*
tcp SYN-RECV 0 0 157.0.0.1:110 157.0.11.1:52843

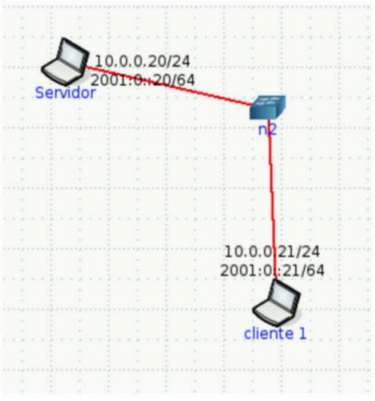
cliente# ss -natu | grep 110
tcp SYN-SENT 0 1 157.0.11.1:52843 157.0.0.1:110
```

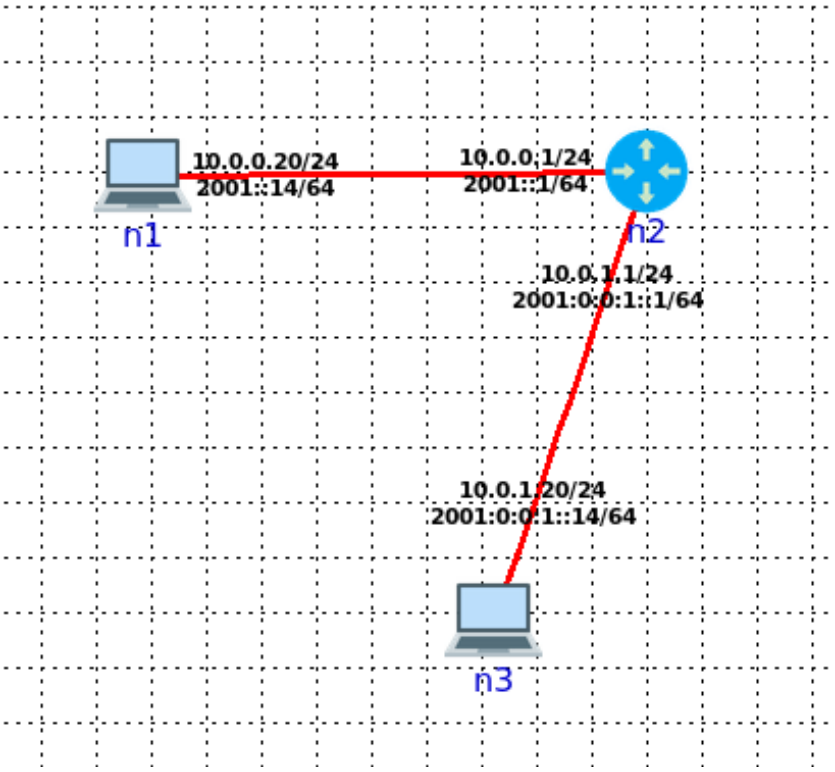
a. **¿Qué segmentos llegaron y cuáles se están perdiendo en la red?**

b. **¿A qué protocolo de capa de aplicación y de transporte se está intentando conectar el cliente?**

c. **¿Qué flags tendría seteado el segmento perdido?**

15. **Use CORE para armar una topología como la siguiente, sobre la cual deberá realizar:**



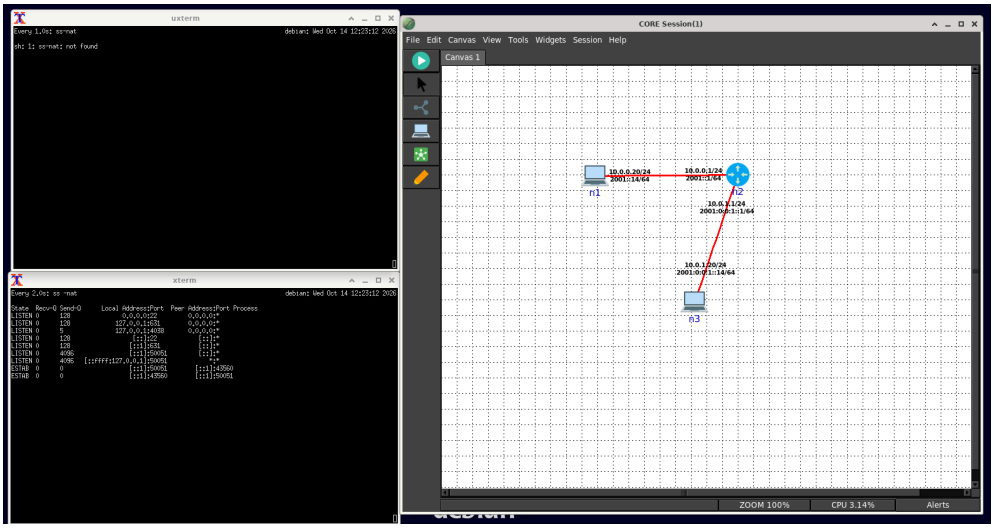


-n → muestra IP's y los numeros de puerto en formato numerico en lugar de nombres de host y servicios

-a → muestra todas las conexiones, incluyendo estado LISTENING y ESTABLISHED

-t → filtra resultados para mostrar solo conexiones TCP

a. **En ambos equipos inspeccionar el estado de las conexiones y mantener abiertas ambas ventanas con el comando corriendo para poder visualizar los cambios a medida que se realiza el ejercicio.**
Ayuda: watch -n1 'ss -nat'.



ME DIO PAJA Y NO SEGUI

b. **En Servidor, utilice la herramienta ncat para levantar un servicio que escuche en el puerto 8001/TCP. Utilice la opción -k para que el servicio sea persistente. Verifique el estado de**

las conexiones.

c. **Desde CLIENTE1 conectarse a dicho servicio utilizando también la herramienta ncat. Inspeccione el estado de las conexiones.**

d. **Iniciar otra conexión desde CLIENTE1 de la misma manera que la anterior y verificar el estado de las conexiones. ¿De qué manera puede identificar cada conexión?**

e. **En base a lo observado en el item anterior, ¿es posible iniciar más de una conexión desde el cliente al servidor en el mismo puerto destino? ¿Por qué? ¿Cómo se garantiza que los datos de una conexión no se mezclarán con los de la otra?**

f. **Analice en el tráfico de red, los flags de los segmentos TCP que ocurren cuando:**

   a. **Cierra la última conexión establecida desde CLIENTE1. Evalúe los estados de las conexiones en ambos equipos.**

   b. **Corta el servicio de ncat en el servidor (Ctrl+C). Evalúe los estados de las conexiones en ambos equipos.**

   c. **Cierra la conexión en el cliente. Evalúe nuevamente los estados de las conexiones.**